

O Mapa Conceitual da Aplicação

Aqui está um resumo de alto nível de tudo o que foi construído, dividido por conceitos fundamentais de engenharia de software e React.

1. A Arquitetura: Páginas, Rotas e Componentes

- **Fundação:** A aplicação foi dividida em "páginas" lógicas (HomePage, PeoplePage, NotFoundPage).
- **Navegação Inteligente (SPA):** Em vez de usar a tag `<a>` que recarrega a página, usamos o `<NavLink>` do `react-router-dom`. Ele apenas muda a URL no navegador e permite que o React decida o que fazer, sem recarregamentos. Isso é o coração de uma *Single Page Application*.
- **O Cérebro do Roteamento (App.tsx):** O componente `<Routes>` atua como um "controlador de tráfego". Ele olha a URL atual e, usando as regras que definimos com `<Route>`, decide qual componente de página deve ser renderizado.
 - `path="/"` mostra a `<HomePage>`.
 - `path="/people"` mostra a `<PeoplePage>`.
 - `path="*" (o coringa)` mostra a `<NotFoundPage>` para qualquer outra URL.

2. O Fluxo de Dados: Da API à Tela

- **Buscando Dados (PeoplePage.tsx):** Usamos o hook `useEffect` para disparar a busca de dados da API assim que o componente é montado.
- **Gerenciando o Estado da Requisição:** Usamos o `useState` para controlar três informações cruciais:
 1. `loading`: Para sabermos quando mostrar o componente `<Loader />`.
 2. `error`: Para sabermos quando mostrar uma mensagem de erro.
 3. `people`: Para guardar a lista de pessoas quando ela finalmente chega da API.
- **Pré-processamento de Dados:** Logo após receber os dados, fizemos um passo importante: "conectamos" os pais. Percorremos a lista de pessoas e, para cada uma, adicionamos as propriedades `mother` e `father`, que são os *objetos completos* dos pais (e não apenas os nomes). Isso simplificou drasticamente a criação dos links na tabela depois.

3. A Fonte da Verdade: A URL

Este é o conceito mais avançado e central da tarefa. Em vez de guardar o que está filtrado ou ordenado em um estado (`useState`), nós guardamos tudo na URL.

- **Lendo da URL (useSearchParams):** Na `PeoplePage`, usamos o hook `useSearchParams` para ler os parâmetros da URL (ex: `?query=john&sex=m&sort=name`).
- **Escrevendo na URL (setSearchParams):** Nos componentes de filtro (`PeopleFilters`) e ordenação (`SortLink`), usamos a função `setSearchParams` para atualizar a URL toda vez que o usuário interage com um filtro.
- **Benefícios:**
 - **Estado Compartilhável:** Você pode copiar a URL, enviar para um colega, e ele verá exatamente a mesma tabela filtrada e ordenada que você.
 - **Navegação Consistente:** O botão "Voltar" do navegador funciona como esperado, desfazendo a última alteração de filtro/ordenação.

- **Código mais Simples:** A PeoplePage não precisa se preocupar em "como" os filtros mudam. Ela apenas lê a URL e reage a ela.

4. Estado Derivado e Performance (useMemo)

- **O Problema:** A lista de pessoas que a tabela realmente mostra (`visiblePeople`) depende da lista original da API e de todos os filtros e ordenações da URL. Fazer esse cálculo (filtrar, depois ordenar) pode ser custoso se a lista for grande.
- **A Solução (useMemo):** Envolvemos todo o cálculo de filtragem e ordenação em um `useMemo`. Isso diz ao React: "Só refaça este cálculo se uma das dependências (a lista `people` original ou os parâmetros da URL) mudar".
- **Resultado:** Se o usuário fizer algo que não muda a lista (por exemplo, redimensionar a janela), o cálculo pesado não é refeito desnecessariamente. É uma otimização de performance crucial.

5. Componentes Reutilizáveis e Inteligentes

Em vez de colocar toda a lógica em um único lugar, nós a quebramos em componentes menores e mais inteligentes.

- **<SearchLink />:** Um "super-link". Ele sabe como pegar os parâmetros atuais da URL e adicionar/modificar/remover apenas os que ele precisa, sem apagar os outros. Ele usa a nossa função `getSearchWith` para fazer a mágica.
- **<SortLink />:** Um especialista em ordenação. Ele usa o `<SearchLink />` por baixo dos panos e adiciona a lógica de:
 1. Verificar se a coluna já está ordenada.
 2. Decidir se a próxima ordenação deve ser ascendente, descendente ou se deve ser removida.
 3. Mostrar o ícone correto (`fa-sort`, `fa-sort-up`, `fa-sort-down`).

Resumo do Fluxo de Interação do Usuário

Para consolidar, veja o que acontece quando o usuário clica em "Male":

1. O usuário clica no `<SearchLink params={{ sex: 'm' }}>`.
2. O `SearchLink` chama `setSearchParams`, atualizando a URL para `.../people?sex=m` (mantendo outros parâmetros como `sort` ou `query`, se existirem).
3. A `PeoplePage` re-renderiza porque a URL (via `useSearchParams`) mudou.
4. O `useMemo` detecta que sua dependência `sex` mudou de `' '` para `'m'`.
5. O `useMemo` refaz o cálculo: ele pega a lista de pessoas, aplica o filtro de sexo e qualquer outro filtro/ordenação ativo.
6. A `PeopleTable` recebe a nova lista `visiblePeople` e a renderiza na tela.

Espero que este mapa ajude a conectar os pontos! Agora você pode olhar para qualquer arquivo e entender qual o seu papel nesta arquitetura.

Qual "bairro" deste mapa você gostaria de explorar com mais detalhes agora?